

Q1

Problem Statement

Given the following pseudocode:

```
#define DEBUG 0  
  
IF DEBUG  
  
PRINT intermediate_value  
  
END IF
```

- Identify the error in directive usage.
- Correct the logic.

Description

This program is intended to print an intermediate value only when debugging mode is enabled. Since `DEBUG` is defined as `0`, the print statement should not execute. The correct approach is to use preprocessor directives instead of an `IF` statement.

Input

No input is required.

Output

No output is displayed because `DEBUG` is disabled.

Correct Pseudocode

```
#define DEBUG 1  
  
#ifdef DEBUG  
  
PRINT intermediate_value  
  
#endif
```

Explanation

- `#define` is a preprocessor directive.
- `#ifdef` checks whether `DEBUG` is defined.
- If `DEBUG` is defined, the intermediate value is printed.
- Otherwise, nothing is printed.

Q2

Problem Statement

```
#define PI _____
```

```
DECLARE radius AS float
```

```
DECLARE area AS float
```

```
INPUT radius
```

```
area = _____
```

```
PRINT area
```

Fill in: Constant ,Formula

Description

This program calculates the area of a circle using the radius entered by the user.

Input

Radius of the circle.

Output

Area of the circle.

Constant

```
#define PI 3.14
```

Formula

```
area = PI * radius * radius
```

Complete Pseudocode

```
#define PI 3.14
```

```
DECLARE radius AS float
```

```
DECLARE area AS float
```

```
INPUT radius
```

```
area = PI * radius * radius
```

```
PRINT area
```

Sample Input

5

Sample Output

78.5

Explanation

The program reads the radius, applies the formula:

$$\text{Area} = \pi \times r \times r$$

and prints the calculated area.

Q3

Problem Statement

```
#define RATE 1.5
```

```
DECLARE x AS int = 10
```

```
DECLARE result AS float
```

```
result = x * RATE
```

```
PRINT result
```

Questions:

- What is the output?
- Why is result declared as float?

Description

This program multiplies an integer value by a floating-point constant and stores the result.

Input

No input is required.

Output

15.0

Explanation

Given:

$x = 10$

$\text{RATE} = 1.5$

Calculation:

$\text{result} = 10 \times 1.5$

$\text{result} = 15.0$

Since the value 1.5 is a floating-point number, the multiplication produces a floating-point result. Therefore, the variable **result** must be declared as **float**.

Q4

Problem Statement

Given the pseudocode:

```
#define MAX 100
```

```
DECLARE temp AS int
```

```
INPUT temp
```

```
IF temp > MAX
```

```
PRINT "Valid"
```

```
ELSE
```

```
PRINT "Invalid"
```

```
END IF
```

Identify the logical error and rewrite the correct pseudocode.

Description

This program checks whether the entered temperature is within the maximum allowable limit. If the temperature is less than or equal to the maximum value, it should be considered valid.

Input

Temperature value.

Output

Displays "**Valid**" or "**Invalid**".

Logical Error

The condition

```
IF temp > MAX
```

```
PRINT "Valid"
```

is incorrect because a temperature greater than the maximum limit should not be considered valid.

Correct Pseudocode

```
#define MAX 100
```

```
DECLARE temp AS int
```

```
INPUT temp
```

```
IF temp <= MAX
```

```
PRINT "Valid"
```

```
ELSE
```

```
PRINT "Invalid"
```

```
END IF
```

Sample Input

80

Sample Output

Valid

Explanation

The program compares the entered temperature with the maximum value. If the temperature is less than or equal to 100, it prints **Valid**; otherwise, it prints **Invalid**.

Q5

Problem Statement

Arrange the following statements in the correct order to convert an amount from one currency to another.

- Define conversion rate
- Input amount
- Calculate converted amount
- Print converted amount

Description

This program converts a given amount into another currency using a predefined conversion rate.

Input

Amount to be converted.

Output

Converted amount.

Correct Order

1. Define conversion rate.
2. Input amount.
3. Calculate converted amount.
4. Print converted amount.

Pseudocode

```
DEFINE conversion_rate
```

```
DECLARE amount AS float  
DECLARE converted_amount AS float  
INPUT amount  
converted_amount = amount * conversion_rate  
PRINT converted_amount
```

Explanation

The conversion rate is defined first. The user enters the amount, the converted value is calculated, and finally the result is displayed.

Q6

Problem Statement

Write a pseudocode to check whether a given value lies between 10 and 50 using predefined constants.

Description

The program checks whether the entered number falls within the specified range.

Input

An integer value.

Output

Displays "**Within Range**" or "**Out of Range**".

Pseudocode

```
#define MIN 10  
#define MAX 50  
DECLARE value AS int  
INPUT value  
IF value >= MIN AND value <= MAX  
PRINT "Within Range"  
ELSE  
PRINT "Out of Range"  
END IF
```

Sample Input

25

Sample Output

Within Range

Explanation

The program compares the entered value with the predefined minimum and maximum values. If the value lies between 10 and 50 (inclusive), it prints **Within Range**; otherwise, it prints **Out of Range**.

Q7

Problem Statement

Write a pseudocode for a billing system that calculates the total bill by applying an optional discount and tax using predefined constants.

Description

The program calculates the total bill based on the price and quantity of items. If a discount is available, it is deducted from the total amount. Then, tax is added to the discounted amount.

Input

- Price of one item
- Quantity of items

Output

- Total Bill Amount

Pseudocode

```
#define DISCOUNT 10
#define TAX 5

DECLARE price AS float
DECLARE quantity AS int
DECLARE total AS float

INPUT price
INPUT quantity

total = price * quantity
total = total - (total * DISCOUNT / 100)
total = total + (total * TAX / 100)

PRINT total
```

Sample Input

Price = 500

Quantity = 2

Sample Output

945

Explanation

The total amount is calculated by multiplying the price with the quantity. A 10% discount is applied first, and then a 5% tax is added. Finally, the total bill is displayed.

Q8

Problem Statement

Find the errors in the following pseudocode and write the corrected version.

```
#define TAX 5%  
  
DECLARE total AS int  
  
total = price + (price * TAX)  
  
PRINT total
```

Description

This program calculates the total amount after adding tax.

Errors

1. #define TAX 5% is incorrect because % cannot be used while defining a constant.
2. total should be declared as a float because the result may contain decimal values.

Correct Pseudocode

```
#define TAX 0.05  
  
DECLARE price AS float  
DECLARE total AS float  
  
INPUT price  
  
total = price + (price * TAX)  
  
PRINT total
```

Input

Price

Output

Total amount after adding tax.

Explanation

The tax value should be represented as 0.05 instead of 5%. Using a float variable ensures accurate decimal calculations.

Q9

Problem Statement

Write a pseudocode to convert temperature from Celsius to Fahrenheit using a predefined constant.

Description

The program reads the temperature in Celsius and converts it into Fahrenheit.

Input

Temperature in Celsius.

Output

Temperature in Fahrenheit.

Formula

$$\text{Fahrenheit} = (\text{Celsius} \times 9 / 5) + 32$$

Pseudocode

```
#define OFFSET 32  
DECLARE celsius AS float  
DECLARE fahrenheit AS float  
INPUT celsius  
fahrenheit = (celsius * 9 / 5) + OFFSET  
PRINT fahrenheit
```

Sample Input

25

Sample Output

77

Explanation

The program accepts the Celsius value, applies the conversion formula, and displays the equivalent Fahrenheit temperature.

Q10

Problem Statement

Write a pseudocode to find the sum and average of five numbers stored in an array.

Description

The program accepts five numbers, calculates their sum, and then finds the average.

Input

Five integer values.

Output

- Sum
- Average

Pseudocode

```
#define SIZE 5

DECLARE arr[SIZE]

DECLARE sum AS int = 0

DECLARE average AS float

DECLARE i

FOR i = 0 TO SIZE-1
    INPUT arr[i]
END FOR

FOR i = 0 TO SIZE-1
    sum = sum + arr[i]
END FOR

average = sum / SIZE

PRINT sum

PRINT average
```

Sample Input

10
20
30
40
50

Sample Output

Sum = 150
Average = 30

Explanation

The program stores five numbers in an array. It adds all the elements to obtain the sum and then divides the sum by the total number of elements to calculate the average.

Conclusion

In this assignment, pseudocode was written for various programming problems involving preprocessor directives, constants, arithmetic operations, conditional statements, arrays, and calculations. Errors in pseudocode were identified and corrected, and appropriate inputs, outputs, and explanations were provided for each question. These exercises help in understanding the fundamentals of program design before implementing programs in the C programming language.